

Module 7: Supervised Learning Fundamentals

Linear Regression, Logistic Regression & Optimization

Data Science Educational Series

Abstract

Welcome! This module is designed to demystify the mathematical engines behind two of the most popular machine learning algorithms. We will move beyond the "black box" approach and understand the "why" and "how" of the equations, complete with visualizations, step-by-step case studies, and comprehensive evaluation metrics.

Contents

1 The Big Picture: Supervised Learning	2
2 Part 1: Linear Regression	2
2.1 Motivation	2
2.2 The Mathematical Model	2
2.3 The Cost Function (How wrong are we?)	2
2.4 Visualization: The Best Fit Line	3
2.5 Gradient Descent: The Learning Algorithm	3
2.5.1 Deep Dive: Deriving the Gradient with Calculus	3
2.6 Study Case 1: Predicting House Prices	4
3 Part 2: Logistic Regression	5
3.1 Motivation	5
3.2 The Sigmoid Function (Activation)	5
3.3 The Logistic Cost Function	5
3.4 Gradient Descent for Logistic Regression	6
3.5 Study Case 2: Pass or Fail?	6
4 Part 3: Evaluation Metrics	7
4.1 Regression Metrics (Continuous Output)	7
4.2 Classification Metrics (Discrete Output)	7
4.2.1 The Confusion Matrix	7
4.2.2 Derived Metrics	8
4.3 Study Case 3: Medical Diagnosis Evaluation	8
4.3.1 Step-by-Step Calculation	8
5 Part 4: Optimization & Calculus Deep Dive	10
5.1 The Derivative as a Compass	10
5.2 Batch vs. Stochastic Gradient Descent (SGD)	10
5.3 Visualizing the Optimization Paths	11
6 Summary & Key Takeaways	11

1 The Big Picture: Supervised Learning

Imagine you are teaching a child to recognize fruits. You show them an apple and say "Apple." You show a banana and say "Banana." Eventually, the child learns to associate the *features* (color, shape) with the *label* (name).

Key Definitions

- **Supervised Learning:** The machine learns from labeled training data. We know the correct answers (y) for our training examples (x).
- **Regression:** Predicting a continuous value (e.g., Price, Temperature, Speed).
- **Classification:** Predicting a discrete category (e.g., Yes/No, Spam/Ham, Cat/Dog).

2 Part 1: Linear Regression

2.1 Motivation

You want to sell your car. You know that as a car gets older, its price drops. You have data on 5 friends who sold their cars recently. Can we find a mathematical line that best fits this data to predict your car's price?

2.2 The Mathematical Model

In high school, you learned the equation of a line: $y = mx + c$. In Machine Learning, we use fancy Greek letters, but it is the exact same logic!

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

- $h_{\theta}(x)$: The **Hypothesis** (our prediction).
- x : The input feature (e.g., Car Age).
- θ_1 : The **Weight** (slope). How much does price change per year of age?
- θ_0 : The **Bias** (intercept). What is the base price when age is 0?

2.3 The Cost Function (How wrong are we?)

We need a way to measure the error. We can't just add up the errors (*prediction* – *actual*) because negatives and positives might cancel each other out. Instead, we square the errors.

Deriving Mean Squared Error (MSE)

1. The Error: Difference between predicted value and actual value.

$$Error^{(i)} = h_{\theta}(x^{(i)}) - y^{(i)}$$

2. Squared Error: Square it to punish large errors more severely.

$$(h_{\theta}(x^{(i)}) - y^{(i)})^2$$

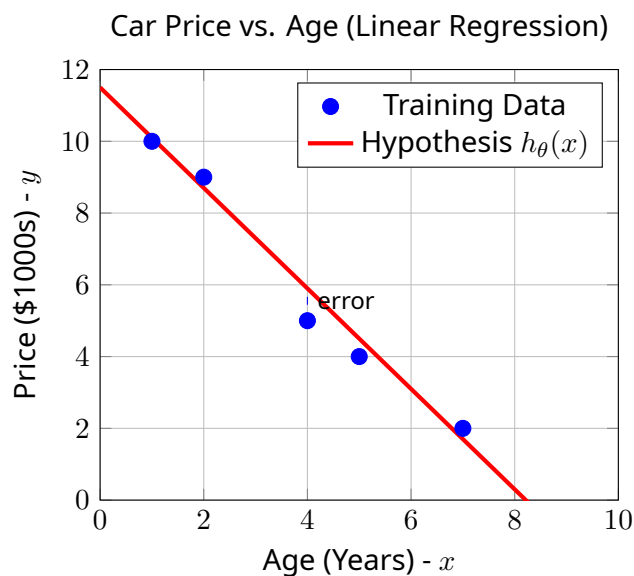
3. The Average (with a twist): Sum errors for all m training examples and average them.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(h_{\theta}(x^{(i)}) - y^{(i)} \right)^2$$

Why the $\frac{1}{2}$? When we take the derivative later, the exponent 2 comes down ($2 \times \frac{1}{2} = 1$). This "foresight" simplifies our math later!

Goal: Find θ_0 and θ_1 that minimize $J(\theta)$.

2.4 Visualization: The Best Fit Line



2.5 Gradient Descent: The Learning Algorithm

Imagine you are standing on top of a mountain (high Cost) in dense fog. You want to get to the valley (low Cost). You feel the slope of the ground and take a step downhill.

2.5.1 Deep Dive: Deriving the Gradient with Calculus

How do we actually get the formula? We apply the **Chain Rule** to find the partial derivative of the Cost Function $J(\theta)$ with respect to each parameter.

Calculus Derivation Breakdown

The Objective: Minimize $J(\theta) = \frac{1}{2m} \sum (h_{\theta}(x) - y)^2$.

Step 1: The General Derivative (Chain Rule) Recall Power Rule: $\frac{d}{du}(u^2) = 2u \cdot u'$.

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{2m} \sum 2(h_{\theta}(x) - y) \cdot \frac{\partial}{\partial \theta_j}(h_{\theta}(x) - y)$$

The 2 and $\frac{1}{2}$ cancel out!

Step 2: Inner Derivative for Bias (θ_0) Here, $h_{\theta}(x) = \theta_0 + \theta_1 x$. The derivative of θ_0 is just 1.

$$\frac{\partial}{\partial \theta_0}(\theta_0 + \dots) = 1$$

So, the gradient for θ_0 is:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot 1$$

Step 3: Inner Derivative for Weight (θ_1) Here, we derive $\theta_1 x$. The derivative with respect to θ_1 is x .

$$\frac{\partial}{\partial \theta_1} (\dots + \theta_1 x) = x$$

So, the gradient for θ_1 is:

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

2.6 Study Case 1: Predicting House Prices

Dataset: Tiny Housing Market

House	Size (x) [1000 sqft]	Price (y) [\$100k]
1	1	1
2	2	2
3	3	3

Initial Guess: $\theta_0 = 0, \theta_1 = 0$. (Line is horizontal at $y = 0$).

Learning Rate: $\alpha = 0.1$

Iteration 1 Calculation:

1. Calculate Predictions ($h_{\theta}(x)$):

- $x = 1 \rightarrow h(1) = 0 + 0(1) = 0$. Error = $0 - 1 = -1$.
- $x = 2 \rightarrow h(2) = 0 + 0(2) = 0$. Error = $0 - 2 = -2$.
- $x = 3 \rightarrow h(3) = 0 + 0(3) = 0$. Error = $0 - 3 = -3$.

2. Calculate Gradient for θ_1 (Slope):

$$\begin{aligned} \frac{1}{3} \sum (Error \cdot x) &= \frac{1}{3} [(-1 \cdot 1) + (-2 \cdot 2) + (-3 \cdot 3)] \\ &= \frac{1}{3} [-1 - 4 - 9] = \frac{-14}{3} \approx -4.66 \end{aligned}$$

3. Update θ_1 :

$$\theta_1 := 0 - (0.1 \times -4.66) = \mathbf{0.466}$$

Result: The slope increased from 0 to 0.466. It is moving towards the correct answer (which is clearly 1).

3 Part 2: Logistic Regression

3.1 Motivation

Linear regression is great for numbers, but what if we want to predict: *Is this tumor malignant (1) or benign (0)?*

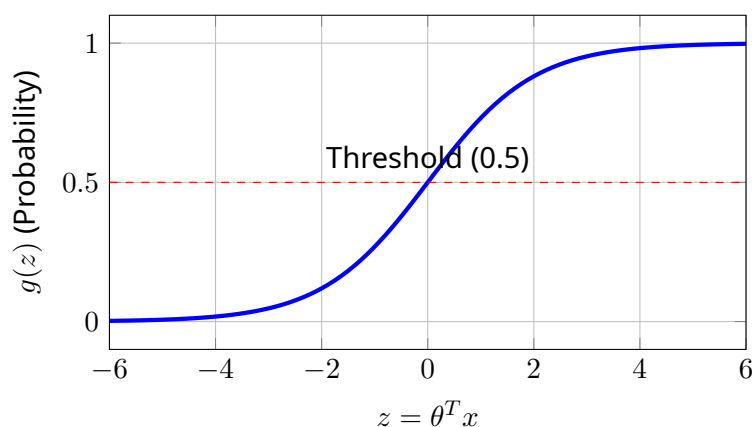
If we use Linear Regression, the line might predict 0.2, 0.8 (which is fine), but also 1.5 or -0.3 . These numbers don't make sense as probabilities. We need to "squash" the output between 0 and 1.

3.2 The Sigmoid Function (Activation)

We wrap our linear equation $z = \theta^T x$ inside the **Sigmoid Function** $g(z)$:

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

The Sigmoid Function



Decision Boundary

- If $h_{\theta}(x) \geq 0.5$, predict $y = 1$.
- If $h_{\theta}(x) < 0.5$, predict $y = 0$.

3.3 The Logistic Cost Function

Problem: If we used Mean Squared Error (MSE) here, the combination of the non-linear Sigmoid function and the squared error would create a "wavy" cost function with many local minima. Gradient Descent would likely get stuck!

Solution: We use **Log Loss** (Cross-Entropy), which ensures the cost function is **Convex** (bowl-shaped).

Log Loss Formula Construction

Case 1: True label is 1 ($y = 1$) We want $h_{\theta}(x) \approx 1$.

$$Cost = -\log(h_{\theta}(x))$$

Why? $\log(1) = 0$ (Zero cost if correct). $\log(0) \rightarrow -\infty$ (Infinite cost if wrong). The negative sign flips this to $+\infty$.

Case 2: True label is 0 ($y = 0$) We want $h_{\theta}(x) \approx 0$.

$$Cost = -\log(1 - h_{\theta}(x))$$

Why? If $h = 0$, $\log(1) = 0$ (Zero cost). If $h = 1$, $\log(0) \rightarrow -\infty$ (Infinite cost).

Merged Equation (Binary Cross Entropy):

$$J(\theta) = -\frac{1}{m} \sum_{i=1}^m \left[\underbrace{y^{(i)} \log(h_{\theta}(x^{(i)}))}_{\text{Active if } y=1} + \underbrace{(1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))}_{\text{Active if } y=0} \right]$$

3.4 Gradient Descent for Logistic Regression

The Calculus Miracle

It turns out that if you take the derivative of the complicated Log Loss function with the Sigmoid inside it, the math simplifies beautifully!

Step 1: Derivative of Sigmoid $g'(z) = g(z)(1 - g(z))$. This is a special property of sigmoid.

Step 2: Derivative of Log Loss Using Chain Rule on the Log Loss formula involving logarithms and sigmoid... lots of terms cancel out (e.g., the fraction denominators in derivative of log cancel with the numerator in derivative of sigmoid).

Step 3: The Result

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Look familiar? This is **identical** to the Linear Regression update rule! The only difference is that $h_{\theta}(x)$ is now the Sigmoid function, not just $\theta^T x$.

3.5 Study Case 2: Pass or Fail?

Dataset: Exam Study Hours

Student	Hours (x)	Result (y)	Initial Prediction
Alice	1	0 (Fail)	$z = -1 \rightarrow h = 0.27$
Bob	2	0 (Fail)	$z = 0 \rightarrow h = 0.50$
Charlie	4	1 (Pass)	$z = 2 \rightarrow h = 0.88$

Assume current model: $\theta_0 = -2, \theta_1 = 1$. So $z = -2 + 1x$.

Let's calculate the cost for **Charlie** ($y = 1, x = 4$):

1. ****Calculate z :**

**** $z = -2 + 1(4) = 2$.**

2. ****Calculate Hypothesis:**

**** $h(x) = \frac{1}{1+e^{-z}} \approx 0.88$.**

3. ****Calculate Cost:**

**** Since $y = 1$, use $-\log(h(x))$.**

$$Cost = -\log(0.88) \approx 0.12$$

The cost is low because the prediction (0.88) was close to the actual value (1).

4 Part 3: Evaluation Metrics

How do we know if our model is "good"? A low cost function is great during training, but we need interpretable metrics to report to humans.

4.1 Regression Metrics (Continuous Output)

When we predict house prices, we don't care about "accuracy" (a house price is rarely predicted *exactly* to the cent). We care about how far off we are.

MSE vs. MAE

1. MSE (Mean Squared Error):

$$MSE = \frac{1}{m} \sum_{i=1}^m (y^{(i)} - h(x^{(i)}))^2$$

Interpretation: Squares the errors. If you have one huge error (e.g., predicting a \$1M house as \$50k), MSE explodes. **Use when:** Large errors are unacceptable.

2. MAE (Mean Absolute Error):

$$MAE = \frac{1}{m} \sum_{i=1}^m |y^{(i)} - h(x^{(i)})|$$

Interpretation: The average absolute distance. "On average, our prediction is off by \$5,000." **Use when:** You want a metric that is easy to explain to non-technical stakeholders.

4.2 Classification Metrics (Discrete Output)

In classification, "Accuracy" can be misleading. Imagine a dataset where 99% of patients do **not** have cancer. A model that predicts "No Cancer" for everyone is 99% accurate but completely useless!

4.2.1 The Confusion Matrix

This is the foundation of all classification metrics. It breaks down predictions into 4 boxes.

		Actual Class	
		Positive (1)	Negative (0)
Predicted Class	Positive (1)	True Positive (TP) (Correct Hit)	False Positive (FP) (False Alarm / Type I)
	Negative (0)	False Negative (FN) (Missed / Type II)	True Negative (TN) (Correct Rejection)

4.2.2 Derived Metrics

Key Metric Formulas

1. Accuracy: Overall correctness.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2. Precision: Of all the times the model shouted "Positive!", how many were right?

$$Precision = \frac{TP}{TP + FP}$$

High stakes: Spam detection (You don't want to lose important emails in spam).

3. Recall (Sensitivity): Of all the *actual* positive cases, how many did we find?

$$Recall = \frac{TP}{TP + FN}$$

High stakes: Cancer diagnosis (You cannot afford to miss a sick patient).

4. F1 Score: Harmonic mean of Precision and Recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

4.3 Study Case 3: Medical Diagnosis Evaluation

Dataset: 5 Patients

We built a model to detect a disease. 1 = Sick, 0 = Healthy.

Predictions vs Actuals:

- **Patient A:** Actual: 1, Predicted: 1 (**TP**)
- **Patient B:** Actual: 1, Predicted: 0 (**FN - Dangerous Miss!**)
- **Patient C:** Actual: 0, Predicted: 0 (**TN**)
- **Patient D:** Actual: 0, Predicted: 1 (**FP - False Alarm**)
- **Patient E:** Actual: 0, Predicted: 0 (**TN**)

4.3.1 Step-by-Step Calculation

1. Build the Confusion Matrix:

- TP = 1 (Patient A)
- FN = 1 (Patient B)
- FP = 1 (Patient D)
- TN = 2 (Patients C, E)
- **Total = 5**

2. Calculate Accuracy:

$$\frac{TP + TN}{Total} = \frac{1 + 2}{5} = \frac{3}{5} = \mathbf{60\%}$$

3. Calculate Precision (Trustworthiness):

$$\frac{TP}{TP + FP} = \frac{1 + 1}{2} = \frac{1}{2} = \mathbf{50\%}$$

Meaning: When model predicts sick, it's a coin flip.

4. Calculate Recall (Coverage):

$$\frac{TP}{TP + FN} = \frac{1}{1 + 1} = \frac{1}{2} = \mathbf{50\%}$$

Meaning: We missed half the sick people.

5. Calculate F1 Score:

$$2 \times \frac{0.5 \times 0.5}{0.5 + 0.5} = 2 \times \frac{0.25}{1.0} = \mathbf{0.50}$$

5 Part 4: Optimization & Calculus Deep Dive

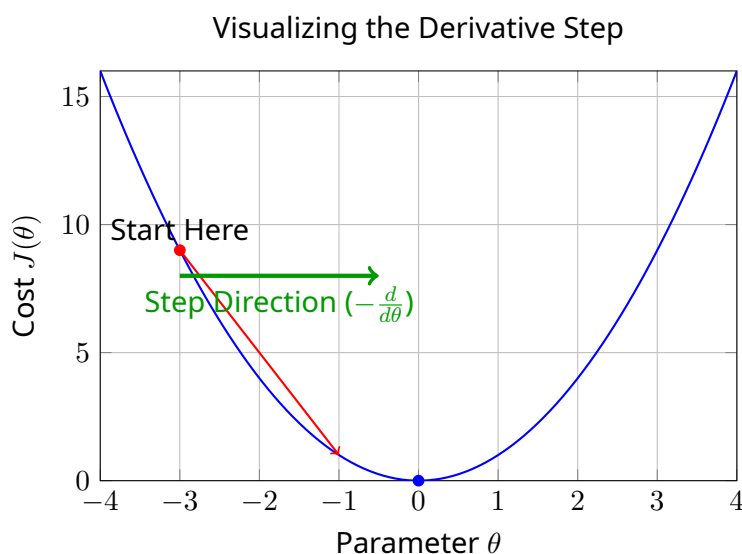
We've used Gradient Descent, but let's zoom in on the *calculus engine* that drives it and explore how we can make it faster.

5.1 The Derivative as a Compass

In calculus, the derivative $f'(x)$ tells us the **slope** of the tangent line at any point.

- If slope is **positive** (uphill), we must go **left** (decrease x) to find the bottom.
- If slope is **negative** (downhill), we must go **right** (increase x).

This is why the update rule has a minus sign: $\theta_{new} = \theta_{old} - \text{Slope}$.



5.2 Batch vs. Stochastic Gradient Descent (SGD)

The standard Gradient Descent we learned is technically called **Batch Gradient Descent**. It calculates the error for *every single student* (all m examples) before taking one step.

1. Batch Gradient Descent

The Logic: "I will look at all 1,000,000 data points, calculate the average slope, and take one perfect step."

$$\theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Pros: Stable convergence.

Cons: Extremely slow on huge datasets.

2. Stochastic Gradient Descent (SGD)

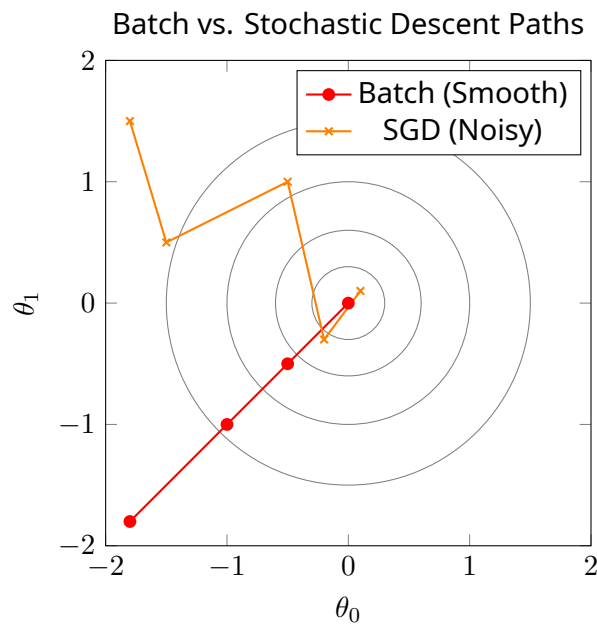
The Logic: "I will look at **one** random data point, calculate the slope, and take a step immediately."

$$\theta_j := \theta_j - \alpha (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Pros: Blazing fast steps.

Cons: The path is "noisy" (drunk walk), bouncing around the minimum.

5.3 Visualizing the Optimization Paths



6 Summary & Key Takeaways

1. **Linear Regression** fits a straight line to predict continuous values. It minimizes the squared differences.
2. **Logistic Regression** uses the S-shaped sigmoid curve to predict probabilities for classification (0 or 1). It minimizes Log Loss.
3. **Gradient Descent** is the workhorse for both. It iteratively tweaks the weights (θ) to slide down the cost mountain.
4. **Evaluation Matters:** Accuracy is often insufficient. Use Precision when false alarms are bad, and Recall when missed detections are dangerous.
5. **Optimization Trade-offs:** Use Batch Gradient Descent for small datasets (stable) and Stochastic Gradient Descent (SGD) for massive datasets (fast).